CYBERSECURITY TASK 3 2026

By Hudi Juma Sendoro

API SECURITY ANALYSIS

Date: 26/03/2026

---

1. Introduction

An API (Application Programming Interface) acts as a middleman between a server and a client, enabling users to retrieve and interact with data efficiently. For this analysis, I used Postman to retrieve information from a sample public API and uncover vulnerabilities that a poorly configured API can introduce — both to businesses and individuals.

API security is critical because APIs serve as the front door to sensitive data. When left unsecured, they can allow attackers to steal user information, impersonate legitimate users, or overwhelm a system with spam requests — ultimately crashing services entirely.

The goal of this analysis was to identify real API vulnerabilities, understand their potential impact, and suggest practical remediation steps that could be applied in a real-world environment.

---

2. Methodology

This analysis was conducted using Postman, a widely used API testing tool. The target was the Postman Echo API (postman-echo.com), a public demo API designed for testing and educational purposes.

The following security aspects were examined:

- Exposure of sensitive data through response headers
- Sensitive data leakage via URL parameters
- Authentication and token validation weaknesses

- Rate limiting controls
- Input validation and susceptibility to code injection (XSS)

Each test was performed manually by crafting specific requests and observing how the API responded under each condition.

---

3. Risk Summary

#	Finding	Severity
1	Sensitive Data Exposed in Response Headers	Medium
2	Sensitive Data Exposed in URL Parameters	High
3	Broken Authentication — Invalid Token Accepted	Critical
4	No Rate Limiting	High
5	No Input Validation / XSS Vulnerability	High

---

4. Findings

Finding #1: Sensitive Data Exposed in Response Headers

Severity: Medium

- During the initial GET request to the Postman Echo API, the server returned response headers that exposed sensitive infrastructure information. This included session cookies (Set-Cookie), the server provider (Cloudflare), and internal request identifiers (CF-RAY).
- Session cookies are dangerous as they represent active user sessions, if intercepted, an attacker could use them to impersonate a legitimate user without ever needing their password. For Instance: This is similar to someone finding your hotel key card in the hallway they never checked in, but they now have full access to your room.

Impact: Exposes server infrastructure and active session tokens to anyone monitoring network traffic.

Recommendation: Configure the server to suppress unnecessary headers and apply the HttpOnly and Secure flags to all cookies.

Body Cookies 3 Headers 14 Test Results 1/1 200 OK • 337 ms • 1.07 KB • Save Response

Content-Encoding	gzip
etag	W/"f6-oe7nAmlIXKWkkWPw42OJ0UjI2Pw"
vary	Accept-Encoding
Set-Cookie	sails.sid=s%3AUOsiHJqb5rp7R0-vOzTpvTo0iw7WKMRC.62CxQsO
Set-Cookie	__cf_bm=jZieZBJClbOCjS.I89WXt4p.ZukP52ImLJKFXmSpJ0Y-1774
Set-Cookie	_cfuvid=m_JKIfMd8899FiVrsfWR4oR_OnNXimLeR9HUZMhHvB4-17
x-envoy-upstream-service-time	5
cf-cache-status	DYNAMIC
Server	cloudflare
CF-RAY	9e0fd486cdd8eeb7-WAW

Finding #2: Sensitive Data Exposed in URL Parameters

Severity: High

- During testing, sensitive credentials, specifically a username and password were passed directly through the URL as query parameters:

`postman-echo.com/get?username=john&password=12345`

- The API accepted this without any warning or rejection, and echoed the credentials back in plain text within the response body. This means the password was fully visible in the URL itself — not hidden or encrypted in any way.
- This is dangerous because URLs are routinely stored in browser history, server logs, and internet provider records. Any system the request passes through has full visibility of those credentials.

Impact: User credentials can be harvested from browser history, server logs or network monitoring tools — without any sophisticated attack required.

Recommendation: Sensitive data such as passwords and tokens should never be passed through URL parameters. They should be sent in the request body using a POST request over HTTPS, ensuring they are not logged or cached anywhere.

Body Cookies 3 Headers 12 Test Results 1/1 200 OK 209 ms 1.01 KB Save Response

{} JSON Preview Visualize


```
3        "username": "john",
4        "password": "12345"
5    },
6    "headers": {
7        "host": "postman-echo.com",
8        "accept-encoding": "gzip",
9        "accept": "*/*",
10       "x-forwarded-proto": "https",
11       "user-agent": "PostmanRuntime/7.52.0",
12       "postman-token": "96af3817-ae95-4f42-8fd0-c9bdba3fd955",
13       "cookie": "__cf_bm=jZieZBJCIbOCjS.I89WXt4p.ZukP52ImLJKFXmSpJ0Y-1774294012-1.0.1.1-2ZqDZ4j9So.
             yrWQ2dar4ev8xEsEdr6pQvmFxwQ1.CZawci3HehezJ3bq5zbo_cz9aIERwJkk.bAzDoPQXzrm3dqUxfdRBjVqXwNV.
             vnZwdk; _cfuvid=m_JKlfMd8899FiVrsfWR4oR_OnNXimLeR9HUZMhHvB4-1774294012112-0.0.1.1-604800000;
             sails.sid=s%3AUOsiHJqb5rp7R0-vOzTpvTo0iw7WKMRC.
```

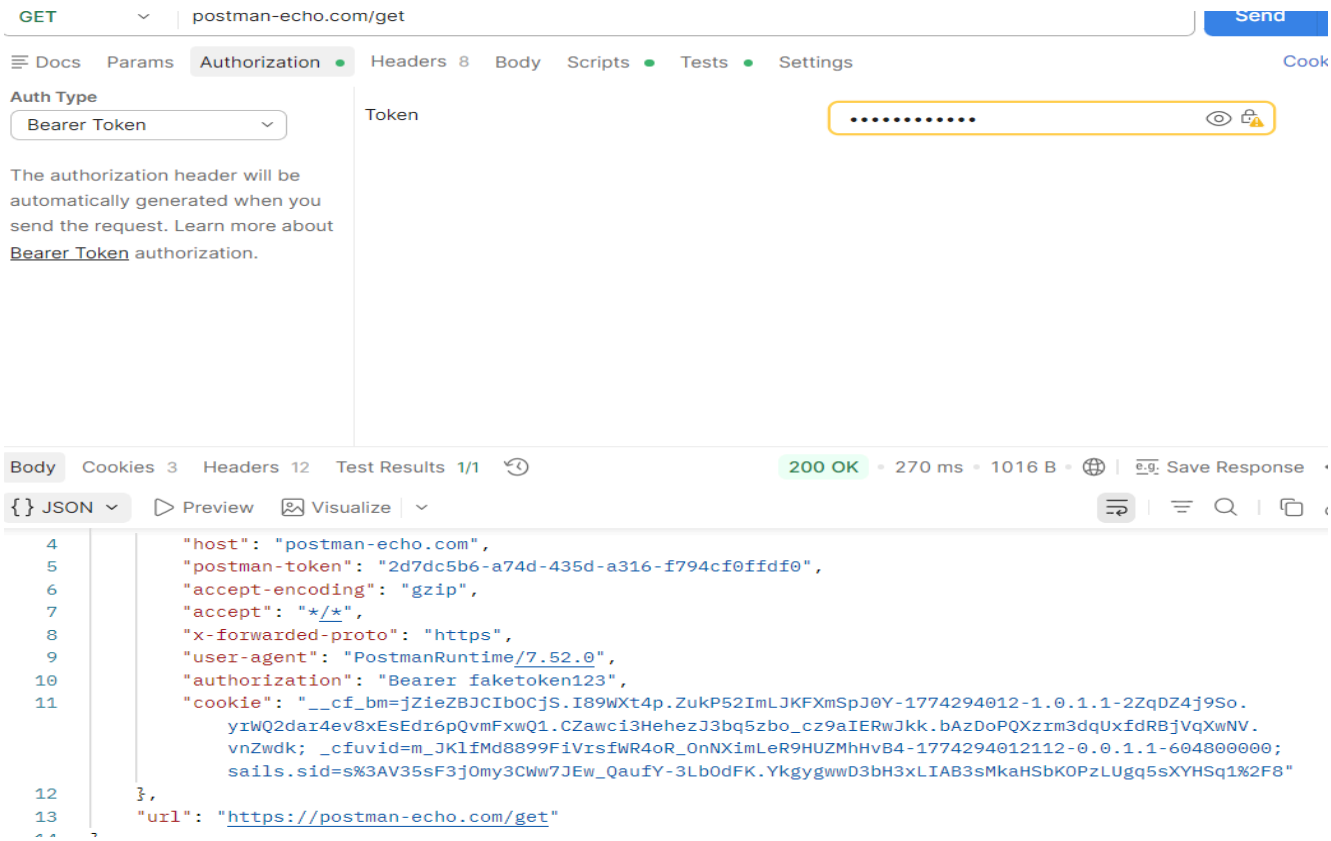
Finding #3: Broken Authentication / Invalid Token Accepted

Severity: Critical

- To test authentication strength, a completely made up token (faketoken123) was submitted as a **Bearer Token** in the Authorization header. A properly secured API should verify whether a token is valid, belongs to a real user, and has not expired before granting access.
- Instead, the API returned a **200** OK response, granting full access without any kind of verification. The fake token was simply echoed back in the response headers as if it were legitimate.
- This means anyone, with no credentials can access the API by submitting any random string as a token. The authentication layer exists in appearance only, providing no real protection.

Impact: Unauthorized users can freely access protected API endpoints, potentially exposing sensitive data or functionality to anyone who sends a request — regardless of identity.

Recommendation: Implement proper token validation on every request. Tokens should be verified against a database or decoded using a secret server-side key. Any invalid, expired or unrecognized token should immediately return a 401 Unauthorized response and terminate the request.



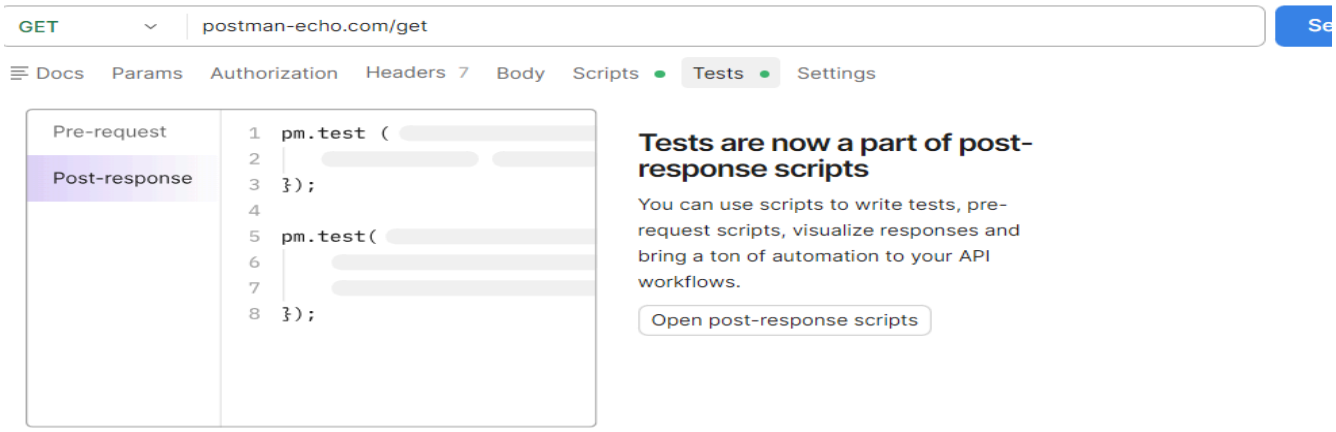
Finding #4: No Rate Limiting

Severity: High

- To test whether the API had any **request handling** in place, multiple rapid requests were sent in quick succession to the same endpoint. A properly secured API should monitor request frequency and block or slow down clients that exceed a reasonable threshold.
- The API **returned 200 OK** on every single attempt without any slowdown, warning, or temporary block. Notably, the number of successive attempts made had absolutely no effect on the API's behavior.It remained fully accessible and responsive throughout, making it directly vulnerable to DDoS attacks.
- This leaves the API wide open to automated attacks that rely on repetition such as brute forcing passwords by trying thousands of combinations and Distributed Denial of service attacks

Impact: Without rate limiting, attackers can automate thousands of requests per second, enabling credential brute forcing, data scraping, and denial of service attacks that could take down the entire system.

Recommendation: Implement rate limiting at the API gateway or server level. A common standard is to allow a maximum number of requests per IP address per minute and return a 4**29 Too Many Requests** response when the threshold is exceeded. Tools like Nginx, Cloudflare, or API Gateway can handle this automatically.



Finding #5: No Input Validation / XSS Vulnerability

Severity: High

To test whether the API validated or sanitized incoming data, a classic Cross Site Scripting (XSS) payload was injected through a URL query parameter:

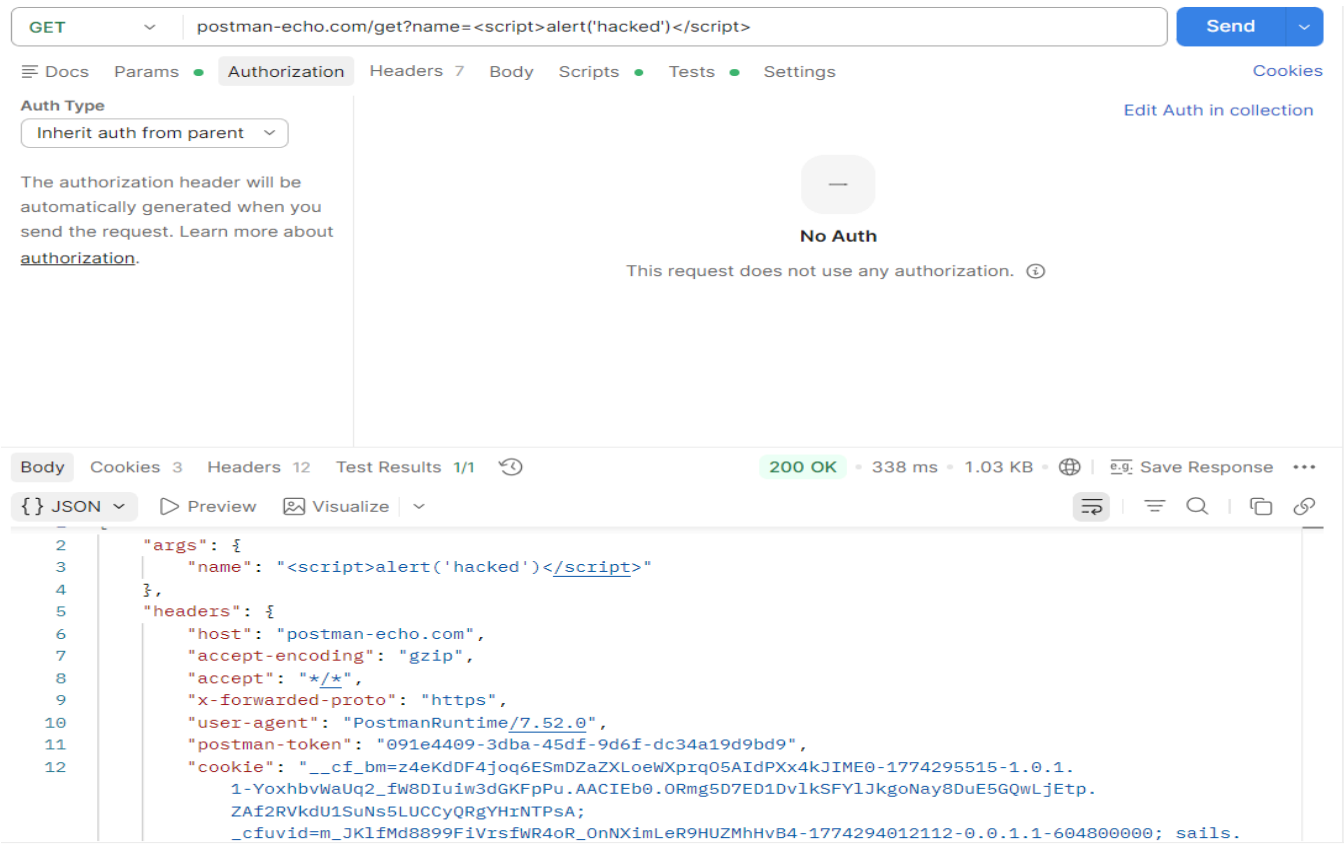
```
postman-echo.com/get?name=<script>alert('hacked')</script>
```

- A properly secured API should immediately recognise this as malicious input, reject it, and return a **400 Bad Request response**. Instead, the API accepted the script tag without any complaint and echoed it straight back in the response body with a 200 OK, treating it as completely normal input. This directly violates OWASP (Open Web Application Security Project) principles, which explicitly list injection attacks and improper input validation among the Top 10 most critical API security risks.

- This means the API has no mechanism to distinguish between legitimate user input and malicious code. As observed during testing, a misconfigured system can blindly accept a malicious JavaScript payload through an input field. If this data were stored and later rendered in a browser, it would execute automatically, giving an attacker the ability to steal session cookies, redirect users to malicious sites, or perform actions on their behalf without their knowledge.

Impact: Malicious scripts can be injected through the API and potentially executed in end users' browsers, leading to session hijacking, credential theft and unauthorized actions.

Recommendation: Implement strict input validation and sanitization on all API endpoints. All incoming data should be checked against expected formats and any HTML or script tags should be escaped or rejected outright. Libraries such as DOMPurify or server side validators can handle this automatically.



---

5. Remediation

Based on the findings, here is what should be fixed and in what order:

1. Fix Authentication First (Critical) There is no point securing anything else if anyone can walk in with a fake token. The API needs to actually **check whether a token is real before granting access**. If it does not recognize it, block it immediately with a 401 Unauthorized response. No exceptions.

2. Stop Putting Passwords in URLs (High) Passwords should never be visible in a URL. full stop. They need to be moved into the request body using a POST request over HTTPS so they never show up in logs, browser history or anywhere else they should not be.

3. Hide Your Infrastructure (Medium) There is no reason for a response to tell the world which server provider you use or hand out session cookies freely. Tighten up those response headers and make sure cookies have Http Only and Secure flags

4. Limit Repeated Requests (High) The API should cut off users who are clearly spamming it. Set a reasonable request limit per IP address and return a 429 Too Many Requests response when it is exceeded — this alone shuts down most brute force and DDoS attempts.

5. Don't Trust User Input (High) Never assume what a user sends is safe. Script tags and suspicious characters should be rejected before they go anywhere near the server. Following OWASP guidelines here is not optional, it is the baseline..

---

6. Conclusion

- The Postman Echo API, as tested, was nowhere near a secure implementation. The vulnerabilities discovered were not sophisticated or deeply hidden.They were fundamental gaps that are simple to identify yet carry serious consequences. The difference between a secured and unsecured API can mean the difference between regulatory compliance and a GDPR violation, between a trusted product and a reputational disaster, between a functioning service and a completely compromised one.
- The bigger picture here goes beyond this single API. Security is not an optional feature to be added later. It is a foundation. Adhering to established standards such as OWASP principles is not just best practice, it is the baseline expectation for any API exposed to real users and real data.
- On a personal note, this analysis was a genuinely eye opening experience. Coming in with little prior knowledge of APIs, the process of actually testing, observing and documenting these vulnerabilities provided a much clearer and more grounded understanding of API security than theory alone ever could. I am not claiming to be an expert, but I now have a solid foundation to build on.